P 前三项优化：出发点、实现方案与验证

文档版本：cf570ea69ff05b009c076b79414629b44a86d346

源文档 SHA-256：88ed0dd1c610a81734b666c614bf140c29635a0fa97fed7a16d75bfb8aeab2e6

原文与链接：查看固定版本来源

转换说明：保留全部正文、表格、代码与证据边界；未添加新的 token/s 结论。

关联 Issue247、PR243。负责人 Melodydt；2026-11-09，Asia/Shanghai。本轮由研究进入原生 RTL 的第一阶段实现，起点为 `3a66f8de0889925846e9d737690e2d9e297bb173`。独立审查、物理验收和整模型部署尚未完成。

1. 要解决什么问题

前三项是：P 供数/预取与消费者仲裁、P 的 S 读写并发、多个部分和上下文配合 A 复用。共同目标是减少 M 等待数据和重复装载的时间，而不是增加乘法器数量。允许增加控制逻辑、缓冲和寄存器，但本轮仍保持每个 P 的 S 为 **16KiB**；48/64KiB 扩容只保留在既有方案中。

此前 P/DMA 研究、面积换速度研究和容量/端口对照提供方向选择依据：同一条件模型的 W8/context512，优化后的供数基线为 3.063655 token/s，理想 P 1R1W 加八组上下文为 3.496169 token/s。但那是 PR241 的 S32、候选端口/累加器合同及条件时钟下的仿真，不能作为原生 P 已获得的速度。

原生代码实际具备的资源如下：

资源	现状	对实现的约束
S_P	四个 256×128、单端口 1RW bank，总计 16KiB	每 bank 每拍最多一次访问；512bit 读占四 bank
普通 S 入口	两个外部 IO、本地 G、M 写回，汇聚为一个 128bit 请求口	请求类型不能仅凭读写方向认定为 DMA 预取
feeder	S512→A128/B512；A 两槽、B 一槽；原 LOAD 总会装 A	A 保留需要显式 B-only LOAD
M	S8/S16 输入、S40 段结果、八行×八列 S64 累加	八组上下文可复用已有结果槽，不能套用模型的 96bit 寄存预算
命令/完成	ready/valid，完成保持至 take，写回等真实 S ACK	不能把入队、发请求或 M 运算结束当作最终写完成

2. 优化一：消费者优先与有界公平

出发点

外部写入下一块数据，与 G 读取当前操作数、M 写回已完成结果，都经过普通 S 仲裁器。完全轮转不区分正在阻塞计算链的请求。若当前结果迟迟不能退休，后续计算可能因资源占用而停顿；优先解除这类依赖有机会缩短关键路径。

本轮实现

新增 `popt_s_policy_arbiter`，沿用原仲裁器端口。P 的 client0/1 为外部 IO，client2/3 为 G/M 本地消费者，`CONSUMER_MASK=4'b1100`。启用后两个类别内部各自轮转；两类同时有请求时，连续消费者授权最多 `MAX_CONSUMER_BURST=2` 次，随后必须服务一个外部请求。计数以实际获得服务的事务为准，不能将反复停顿的请求当作已经服务。

请求一旦选定，在 memory ready 前保持 owner、地址、读写和数据；响应在 owner 接收前保持，不能为追逐更高优先级而抢走在途事务。连续授权计数在 `req_valid && req_ready` 时推进，一次停顿不会重复计数；单owner直到响应退休。默认策略仍使用原轮转仲裁器。P 顶层参数 `S_CONSUMER_PRIORITY=1` 才启用新策略。

公平性的单位是**请求授权次数**。若接收端永久不接响应，或一个请求被活动 LOAD 的地址保护持续阻塞，仲裁器无法承诺有限墙钟周期。保护区写若已占据普通仲裁器的唯一owner，后面的无冲突请求也会等到它完成，这一队头阻塞尚未消除。

为什么尚不能称为完整自适应 DMA 预取

当前 P 普通 IO 只提供请求/响应，没有“本 tile 消费者” “下一 tile 预取”、A 驻留范围、目标缓冲代次等描述符。直接以 `write=1` 当预取会把输出回写和普通写混为一谈。本轮实现的是可真实接入的仲裁部分。

后续完整预取的具体步骤：

1. 在编译器/运行时给每个 tile 发出 A/B 区间、bank、缓冲槽、代次、依赖及消费者就绪信息；仍保持有限描述符和在途信用。

2. 写入缓冲在最后一个真实 S 写 ACK 后进入 READY；M/feeder 的最终消费确认后才进入 FREE。禁止提前复用或让后一代覆盖前一代。

3. 仅在目标槽可用且不与活动输入/输出冲突时，排队下一块 B；A 全驻留和分段流入分别选择预取策略，避免容量变化意外改变策略。

4. 把消费者需求读与预取请求显式分类，在外存仲裁处也执行公平限制；本地普通 S 仲裁无法替代 DDR 端仲裁。

5. 将描述符开销、ACK、缓存失效、外存通道争用与错误排空接回实际模型/控制程序，再测 token/s。不能直接使用旧模型的百分比作为此 RTL 的收益。

3. 优化二：固定 16KiB 的 bank 并发

出发点

旧 `pu_p_store_control` 在 LOAD 期间关闭所有普通 S 请求，`local_s_p` 也强制分阶段。即使目标地址完全无关，下一批写入、G 读取或 M 写回都只能等待整个 LOAD 完成。单纯扩大 S 无法消除这种前端串行。

本轮实现与数据路径

顶层 `S_BANK_RW=1` 选择新 `block_feed_s16k_bank_rw`。四个实际 1RW 宏保持不变，不用寄存器数组假装双端口 SRAM。

```
四路普通请求 → 有界公平仲裁 → 128bit 请求 -+
                                            +→ 按 bank 分配 → 四个 1RW SRAM
feeder → 512bit 请求 → 带 mask 的拼接读 --+
                      ↓
                完整 512bit 响应 → 原 A/B feeder
```

无竞争时，一次发射同时读取四 bank。普通请求和新宽读争用时，仲裁可给普通请求一个 bank，其余三个 bank 同拍读取；下一拍优先补齐缺失 bank。这样可在同拍完成 **48B 读取 + 16B 普通读/写**，随后补齐 16B。全局每拍总访问上限仍为 64B，绝不是模型中的 64B 读 + 64B 写。

普通请求与宽读响应拥有独立的有界缓冲和在途预留；普通写只有实际宏写完成后才产生 ACK。宽响应须完整拼接后才 valid，背压时数据稳定。未完成的拼接读优先补齐，防止连续普通请求让宽读长期缺一个 bank。无冲突时保留全宽路径，不固定拆成四个串行窄读。

生命周期与冲突处理

新 LOAD 仍先阻止新普通准入，等已接收的普通响应退休，再建立输入租约。LOAD 期间开放无冲突普通请求，但整个 A/B 输入区间禁止普通写；直到 LOAD done 被 take 后释放。`reuse_b` 不读取 B，因而 B 区间为空；B-only LOAD 的 A 区间为空。普通读取不会改变输入，可并行。

后端另锁定正在拼接的四 word 行，阻止同一行写入在不同片段之间穿插，保证 512bit 快照一致。即使上层没有正确分类请求，也不能得到半新半旧的宽响应。复位同时清空在途/响应/租约；未确认的写不能伪造成功回执。

需要特别保留的成本：两槽窄响应、宽响应/拼接寄存器、每 bank 地址选择器、mask、租约边界比较、仲裁控制及布线。增加面积是否划算、最高频率是否下降，需要综合与物理流程另验。该方案可解除前端整段互斥，但不能保证每种 LOAD 更快，尤其宽读密集且普通请求没有重叠机会时。

真正 1R1W 的后续步骤

若需要兑现模型独立读写带宽，必须取得同容量 1R1W 宏或经过验证的替代组织，明确同址读写语义和延迟，然后接入双通道前端、独立完成信用和输入/输出所有权。现有四个 1RW bank 的布线/时序资格不能转移给新宏。本轮不增加该假设，也不扩容。

4. 优化三：八组 S64 部分和与 A 复用

出发点与循环变换

原方案按输出列组执行完整 K：每个列组都重复装入相同 A 分段。A 行超过 S 容量时，这还会造成重复外存读取。八组部分和使 K 分段可置于外层，A 装一次后供八组 B 使用。

```
原方式：for 输出列组 g: for K分段 k: LOAD A[k], B[k,g]; 累加当前结果
新方式：for K分段 k:
          LOAD A[k], B[k,0]; 累加context0
          for g=1..7: B-only LOAD B[k,g]; 累加context[g]
        所有K完成后分别最终舍入、写回、释放context
```

M 的两套 A/B operand bank、A/B 实际握手和乘法器数量不增加。八个 context 复用原八行结果槽，每槽八个 S64；新增控制状态而非新增八套 M。代价是活动 context 与旧八行批处理不能同时占用同一组槽。原有多行 batch/prefill 是否更适合旧方式，要按工作负载选择。

顶层与命令接口

`pu_p_group.M_CONTEXTS=0` 保留旧 M；设为 1、2 或 8 选择 `block_v2_m_context`。普通 A135/B517 接口和旧命令保持。新命令如下，未列位必须为零：

命令	字段	语义
op13	index[5] operand bank，index[2:0] context	执行一个 K 段，index[4:3]=0
op13	data[0]/[1] A/B wide，[2] first，[3] final	first 只用于 FREE 槽；final 声明最后一段
op13	[7:4] lastK，[11:8] N，[14:12] tiles，[17:15] source A row	K 尾 1..8，N 1..8，tiles 1..4，A row 0..7
op13	[33:18] Koffset，[49:34] group tag	offset 单位为 K 元素，连续性及代次检查
op14	index[2:0] context；data[0] abort，[16:1] expected tag	完成后 release，或显式 abort；避免释放错误代次
op3/op5	保留旧 raw/RNE 读接口，以行号选择 context	context 只有 COMPLETE 后可读，禁止读取中间结果
op12	保留原 S40 部分结果诊断	ACTIVE 时仍可按原条件读取诊断值；不得作为最终结果
op6	就绪边界接受的全清命令	清除输入有效性与上下文，产生自己的响应；不取消执行中的命令
reset	整组复位	清除在途执行、上下文和未退休响应

每个 context 保存 FREE/ACTIVE/COMPLETE、nextK、group tag、格式，N 复用原 result_n。first 必须 offset0；续段必须匹配 tag、nextK、格式和 N，nextK 不得 16bit 回绕。计算错误/溢出不得推进生命周期或 nextK，旧 S64 和原有效性保持，允许修正后重试。活动 context 存在时拒绝旧 op10，避免旧八行执行覆盖新槽。

B-only LOAD 的具体做法

开启 context 后，feeder 参数 `ENABLE_REUSE_A=1`。`load_rows=0` 且 `load_reuse_b=0` 表示只装 B，tiles 仍为 1..4。feeder 不发送任何 A beat，也不清除 M 已有 A；B 仍走实际 S 读、缓冲与 M 握手。默认未开启此功能时 rows0 仍非法。

这条命令不凭空制造 A 有效性：随后 op13 必须检查所选 operand bank、A row/tile 的有效位与格式。调用者必须按 K 外层顺序使用同一 A 分段，不能在其他 context 尚需它时覆盖。本轮没有跨命令 A 引用计数或代次保护；M 的 bank 锁只保护当前执行。B-only 不允许与 reuse_b 同时开启，避免“什么都没装却报告成功”。

数值与存储成本

沿用原 S40 分段乘加、scale 对齐、S64 原子提交和最终 RNE/clamp。每个 K 段可以沿原合同使用 scale，但全段必须表达在同一 S64 数值尺度；调用者负责正确量化定义，硬件不会推断尺度。中间段不舍入，不能用 S16 写回再读入替代完整部分和。

计划增加的上下文元数据为 8×(2bit 状态 + 16bit nextK + 16bit tag + 2bit 格式)=288bit，执行选择约37bit，总计325bit；实际综合映射面积未知。数据 S 保持16KiB，结果数据仍为原64×64bit。此前模型的两P额外1344B不适用于这次复用现有槽的原生实现。

5. 代码落点、启用与验证

文件	做法
project/rtl/block_local_s/popt_s_policy_arbiter.sv	新消费者策略、锁定在途 owner、有界公平
project/rtl/pu_leaf_base/pu_p_store_control.sv	策略选择；新 LOAD 边界先排空，活动 LOAD 允许 bank 模式普通请求
project/rtl/s_sram/block_feed_s16k_bank_rw.sv	四个1RW宏的 mask gather、独立响应和冲突控制
project/rtl/s_sram/local_s_p.sv	后端选择、活动输入区间保护、B-only参数
project/rtl/block_feed_v1/block_feed_loader.sv	rows0 B-only 的合法性、B段遍历与完成
project/rtl/block_v2/block_v2_m_context.sv	原算术基础上的context元数据、命令、合法性与原子提交
project/rtl/pu_leaf_base/pu_p_group.sv	三个独立参数与真实P路径集成
project/experiments/p_native_opt/	单项、组合、旧接口回归及可复跑服务器入口

示例启用：

```
pu_p_group #(
  .S_CONSUMER_PRIORITY(1),
  .S_BANK_RW(1),
  .M_CONTEXTS(8)
) p (...);
```

三个参数默认均为0，避免改变既有系统的默认命令解释。新功能需要新源文件进入编译清单。旧固化的控制镜像/物理网表不会因为源参数存在而自动获得功能；母控/局控描述符编译与镜像接入仍需单独工程验收。

验证分四层：① 仲裁地址/数据/owner、消费者延迟与公平；② 真实bank的并发、同址、拼接、队列满/背压/复位及LOAD输入保护；③ 1/2/8 contexts 的独立整数数值、K/N尾、S8/S16组合、连续K、错误重试和生命周期；④ 完整P的原接口与八组×两K段程序，最终RNE后经真实S写ACK再读回。

所有 EDA 编译和仿真通过获授权服务器 Slurm 执行，本地仅作管理/语法/材料检查。性能对照使用相同初始 S 和同一数值结果，分别报告 A/B beat、S宽读、写回次数及实际周期，不将微例周期换算为大模型 token/s。失败尝试也保留源码、日志、退出码及SHA。

6. 服务器实测：两轮均通过，收益有适用范围

全部为真实 Verilator RTL 执行，SRAM 使用固定 SHA 的实际四个 1RW bank 模型。run10源码 `c2359bdeeea83270ab632861eebe6ffc8dcc510e`，作业101756，1:21 COMPLETED0:0；run11源码 `24f720a788328bd8d73e2bf1a3b0502ded9b937b`，作业101758，1:05 COMPLETED0:0。均为c64/node1/4核/UNLIMITED。第二轮只补受影响的local S/完整P计时及B-only边界，**两轮生产RTL完全相同。**

三方向的真实结果

对照	旧实现/调度	新实现/调度	应如何理解
仲裁，消费者平均响应延迟，burst2	14.9375拍	10.8125拍	延迟下降27.62%；这组总工作仍516拍，未提高总吞吐
local S，LOAD + 64个无关写，无人工背压	95拍	71拍	周期下降25.26%，相同工作率+33.80%；完成25次LOAD期间写入
同一local S，额外32拍feeder背压	122拍	71拍	特定背压条件，不能替代上一行常态收益
SRAM后端，64窄事务 + 32宽读	99拍	99拍	总192次bank操作保持；并发32拍、窄请求最长等待32→1，不能冒称总吞吐提高
完整P，8组、K11，含上下文释放	672拍	676拍	**短任务退化0.60%**，保存的A装载不足以抵消释放开销
完整P，8组、K59，含上下文释放	2112拍	2032拍	周期下降3.79%，相同工作率+3.94%
独立M，8组、S8×S8、K256，含释放	1872拍	1664拍	周期下降11.11%，相同工作率+12.5%；不含S/DDR

完整P使用相同初始S、S8输入、N5、两段或八段K、真实最终RNE、16次S写回ACK后逐字读回。K59的A握手64→8、B64保持、S宽读128→72、写回16保持。仅context与context + bank并发的主计时相同，因为该主程序没有把普通请求主动安排到LOAD内；另一个完整P定向用例验证了外层在LOAD期间确实放行无关写及其真实读回，不能把局部并发收益叠加到2032拍上。

配置归因：测试中的`context`同时开启消费者策略与M contexts，`combined`在其上再开启bank并发。因此完整P两行是这套配置的联合结果，不是严格分离的context/policy单项消融；各单项证据应看独立仲裁、内存与M测试。

修正首轮计时口径： run10完整P的672→644只到最后S写ACK，尚未释放context。run11明确加入八次op14的32拍实际P命令成本，得到672→676；旧日志保留，以新口径评估下一批可用时间。K59同样是2000拍输出前缀 + 32拍释放。最终报告不以未释放的644拍作为完整任务加速证据。

独立M的1/2/8 contexts各44组数值场景：1组44例均退化；2组20改善、8持平、16退化；8组28改善、16退化。8组最差短K为288→297拍。K257的S8×S8、S16×S8、S8×S16、S16×S16分别为2016→1801、2544→2329、2808→2593、4392→4177；B工作和数值保持，均包含释放成本。全部44项与归因。这些测试不按模型真实分布加权，不能将其平均数当token/s。

覆盖与来源

- 仲裁12次正向执行、2个实际负对照：损坏返回数据、故意破坏消费者游标。默认逐拍比对原仲裁器；burst2外部最多被跳过5次授权。严格保持/复位/公平验证通过。

- M共15次执行：原版与候选旧接口各三种初态，完整结果JSON相同，均68787检查、96场景、45724总拍；1/2/8 contexts各44场景×三种初态，含S8/S16、K/N尾、K65535回绕拒绝、两类溢出重试、标签/状态/格式/连续K、背压和复位。两个源/流量篡改拒绝通过。

- run10内存5配置×三种初态；run11 local S六配置×三种初态，H0/H32严格匹配。新增B-only共16种tiles/位宽/bank形状与16个最后合法地址，开启时32合法、46非法；关闭时78个请求全部拒绝且零读。单4word块不存在10位可表达的下一对齐越界起点，14个跨界例不把1024截断成0。所有合法B-only均A=0，B读总计120。

- run11完整P六种配置×三初态，原P程序默认/全启用另六次通过；保留SiLU、直连、错误不写S、真实写ACK、复位等原断言。

- 工具：Verilator5.042、GCC11.4、Python3.10.12；容器、工具与bank SHA见证据。新M严格`-Wall` lint、新C++ `-Wall -Wextra -Werror`，新bank严格lint通过。旧生产/宏完整闭包沿既有`-Wno-fatal`警告策略，不能把它表述为所有源码零警告。

- 两轮完整结果SHA分别 `97de764fffa64b9eb151204af74f94e4d53ea4cabd1f5b5e96c292a0e75bdec0`、`0061270c92a28004e87c0eb3dd69dcd3e0bfd0da6ba97d8bec69d89588fd71f2`。服务器全部800 + 629项输出哈希；回收104 + 66项结果逐项核验。仓库保存185份无损轻量原件，独立核对163份已策展结果及29份当前功能源。日志压缩仅为存储，解码后验证原字节SHA。

7. 启用决策与下一步

这三项已经有可启用的RTL与服务器验证，但应按负载选择。先使用有公平界的消费者策略解除关键依赖；对确实有普通IO/计算回写与LOAD重叠的场景启用bank并发；八context优先用于较长K、多列组，短K保留原模式。批量释放/输出即退休的命令设计是减少短任务开销的下一步，必须明确响应与真实S写ACK后的所有权，不可先删释放再声称加速。

接下来依次完成：①真实DMA描述符、缓冲代次与信用接入，形成完整预取策略；②母控/局控能力协商、编译器生成context/B-only程序及整模型数值回放；③对新控制和M候选做综合面积、STA/P&R；④若仍受bank总带宽限制，再评估同容量真实1R1W宏。SRAM扩容仍不实装。

本轮不提供真实权重/KV、生成质量、端到端token/s、功耗或300MHz物理资格。旧P合格包不因默认参数兼容而自动覆盖新功能。项目状态待独立审查，未合并不done；登记PR5 pending，看板unverified/同步pending。详细复跑、当前CI和可恢复状态见证据入口及交接。